

Final Task Distribution – Weather Monitoring System (IoT Project)

Member 1 – Weather Stations + Kafka Producer + Open-Meteo Bonus + Report

Core tasks (from PDF):

- **3. Installation** – Install Docker, pull images (Bitnami Kafka+Zookeeper, Elasticsearch+Kibana).
- **A) Write Weather Station Mock** – Create 10 simulated stations. Each sends a JSON message every 1 second with:
 - `station_id` (Long), `s_no` (auto-increment per station), `battery_status` (low 30%, medium 40%, high 30%), `status_timestamp` (Unix timestamp), `weather` (humidity %, temperature °F, wind_speed km/h).
 - Randomly drop 10% of messages.
- **B) Set up Weather Station to connect to Kafka** – Use Java Kafka Producer API. Send messages to topic `weather-readings`. Confirm with `kafka-console-consumer`.
- **Dockerfile for Weather Station** – Package the station simulator.
- **5. Deliverables – Report** – Write the final report (PDF/Word) containing: architecture overview, setup instructions, screenshots from Member 4, sample Parquet file from Member 3, sample Bitcask directory from Member 2, JFR results from Member 4, challenges, task distribution.

Bonus tasks (from PDF – 6. Bonus):

- **Open-Meteo API integration** – Implement a **Channel Adapter** that fetches real weather data from [Open-Meteo](#) and publishes it to the same Kafka topic (`weather-readings`) using the same JSON schema.

Dependencies: None (can start immediately).

Member 2 – Bitcask (Full) + Enterprise Integration Patterns (Bonus)

Core tasks (from PDF):

- **Bitcask implementation** (Key-value store for latest reading per station) – from section D:
 - Maintain data files, hint files, key directory.
 - On each new weather message, update `station_id` → latest full reading.
 - Implement file merging (merge logic).
 - **CLI client** – create `bitcask_client.sh` with:
 - `--view-all` → CSV file named `timestamp.csv` (all keys + values).
 - `--view --key=SOME_KEY` → print value to stdout.
 - `--perf --clients=100` → 100 threads query all keys, output `timestamp_thread_N.csv` files.
- **Dockerfile for Central Server** – Provide a temporary Dockerfile for the central server (to be used until Member 3's final version). It should include the Bitcask client and processor (though processor is now with Member 3 – you can skip that part).

Bonus tasks (from PDF – 6. Bonus):

- **Enterprise Integration Patterns** – Implement 5-6 patterns from the list (e.g., Dead Letter Channel, Envelope Wrapper, Idempotent Receiver, Polling Consumer, Claim Check, Message Filter). Apply them to the Bitcask workflow or to the data processing pipeline.

Dependencies: Needs only the JSON message schema from Member 1 (can start Bitcask immediately without it).

Member 3 – Central Station Archiving (Parquet) + Kafka Trigger (Rain Detection)

Core tasks (from PDF):

- **Archiving of all Weather Statuses (Parquet)** – from section D:

- Central server consumes all messages from `weather-readings`.
- Batch writes – accumulate 10,000 records before writing to disk.
- Write to **Parquet files** – partition by time (e.g., year/month/day) and station ID.
- Store files in the **shared volume** (used by Member 4 later).
- **C) Implement Raining Triggers in Kafka Processors** – Use Kafka Processor or DSL. Detect humidity > 70%. Publish a special message to topic `raining-alerts`.
- **Dockerfile for Central Server** – Final version that includes both the Parquet writer and the rain detection processor.

Dependencies: Needs the Kafka Producer (topic `weather-readings`) from Member 1. Can start writing Parquet logic using dummy data while waiting.

Member 4 – Kubernetes + ElasticSearch + Kibana + JFR Profiling

Core tasks (from PDF):

- **E) Set up Historical Weather Statuses Analysis** – Connect Parquet files (from Member 3) to ElasticSearch. Index all weather records.
- **Kibana visualizations** – Create dashboards that prove:
 - Battery status distribution (low 30%, medium 40%, high 30%).
 - Dropped messages rate $\approx 10\%$ per station.
 - Take **screenshots** for the report.
- **F) Deploy using Kubernetes** – Write a single `k8s.yaml` that includes:
 - 10 pods for weather stations (Member 1's image).
 - 1 pod for central server (Member 3's image).
 - Kafka + Zookeeper pods (Bitnami images).
 - ElasticSearch + Kibana pods.
 - PersistentVolumeClaim for shared storage (Parquet + Bitcask).
- **G) Profile Central Station using JFR** – Run system for 1 minute. Report:
 - Top 10 classes with highest total memory allocation.
 - Number of GC pauses and max pause duration.
 - List of I/O operations.

Bonus tasks: None (already moved Open-Meteo to Member 1).

Dependencies: Needs Parquet files from Member 3 (can start writing K8s YAML and JFR scripts immediately using dummy images).

How to work without blocking each other (important)

- **Member 1** – Start immediately. Finish stations and producer first, then Open-Meteo. Write the report last (collect from others).
- **Member 2** – Start Bitcask immediately. It's fully independent. Do patterns at the end.
- **Member 3** – You can write the Parquet writer with dummy data. Once Member 1's producer is ready, integrate and add the rain trigger.
- **Member 4** – Write `k8s.yaml` and JFR scripts first. Wait for Member 3's Parquet files to do Elasticsearch and Kibana.

If someone is late – others continue their independent parts. Only final integration (K8s with all images, ES indexing) requires everyone to have finished.

Deliverables summary (what each member gives to Member 1 for the report)

Member	Deliverables
M1	Full report (written), source code of stations + producer + Open-Meteo adapter, Dockerfile for station.
M2	Source code of full Bitcask, CLI scripts, list of applied Enterprise Integration Patterns.
M3	Source code of central consumer + Parquet writer + rain trigger, Dockerfile for central server, one sample Parquet file.
M4	<code>k8s.yaml</code> , Kibana screenshots (battery + drop rate), JFR profiling report.